

Mi az AJAX?

Az **AJAX** (Asynchronous JavaScript and XML) egy olyan technológia, amely lehetővé teszi, hogy egy weboldal **adatokat küldjön és fogadjon a szervertől anélkül, hogy az egész oldalt újratöltené.**

Ennek köszönhetően a felhasználói élmény sokkal gördülékenyebb, mert az oldal csak a szükséges részeket frissíti (pl. egy táblázat vagy lista).

Fő jellemzői:

Aszinkron adatátvitel (az oldal többi része közben működik tovább).

Tipikus adatcsere-formátum: **JSON** (a korai időkben XML volt elterjedt).

Régebben XMLHttpRequest-tel valósították meg, ma már **Fetch API**-val sokkal egyszerűbb.

A Fetch API szerepe

A modern böngészőkben beépített **Fetch API** váltotta le az elavult XMLHttpRequest megoldást. A Fetch **Promise-alapú** működése miatt tisztább, egyszerűbb és hatékonyabb kódot tesz lehetővé.

Miért jobb a Fetch?

Promise-alapú (nincs Callback Hell) — Nem egymásba ágyazott callbackek sorozata kezeli az aszinkron műveleteket, hanem Promise-lánc.

Tisztább, egyszerűbb szintaxis — Könnyen olvasható, logikus kódszerkezet.

Beépített a böngészőkbe — Nincs szükség külső könyvtárra.

Jobb hibakezelés — Egyértelmű módot kínál arra, hogy kezeld a hálózati hibákat (Promise reject).

Promise-alapú működés: részletes magyarázat

A Fetch egyik legfontosabb tulajdonsága, hogy **Promise-t** ad vissza.

Ez azt jelenti, hogy az aszinkron művelet eredménye **a jövőben** érkezik meg — nem blokkolja a kódot.

Egy Promise három állapotban lehet:

1. **Pending** – még folyamatban van a kérés.
2. **Resolved (Fulfilled)** – a kérés sikeresen befejeződött.
3. **Rejected** – hiba történt (pl. hálózati hiba).

Ha a Fetch sikeresen lefut, a `then()` metódus segítségével feldolgozhatod az eredményt.

Ha hiba történik, a `catch()` metódus fogja el.

```
fetch('https://api.example.com/data')
  .then(response => {
    if (!response.ok) {
      throw new Error('HTTP hiba: ' + response.status);
    }
    return response.json();
  })
  .then(data => {
    console.log('Sikeres válasz:', data);
  })
  .catch(error => {
    console.error('Hiba történt:', error);
  });
```

Magyarázat:

- A `fetch()` meghívása után azonnal **visszaad egy Promise-t**.
- A `then(response => ...)` rész akkor fut le, amikor a válasz megérkezik.
- A `response.ok` igaz értéket ad, ha a státuszkód 200–299 között van.
- Ha hibás státuszkód érkezett (pl. 404 vagy 500), manuálisan kell hibát dobni.
- A `catch()` blokk kezeli a hálózati vagy Promise hibákat.

A Response objektum tulajdonságai

A `fetch()` visszatérési értéke egy Response objektum, amely több hasznos mezőt tartalmaz:

Tulajdonság / metódus	Leírás
<code>response.ok</code>	<code>true</code> , ha a státuszkód 200–299 között van.
<code>response.status</code>	A tényleges HTTP státuszkód (pl. 404).
<code>response.statusText</code>	A státuszkód szöveges leírása (pl. "Not Found").
<code>response.json()</code>	Promise-t ad vissza, amely a választ JSON-ná alakítja.
<code>response.text()</code>	Promise-t ad vissza, amely szöveges formátumban adja vissza az adatot.

Fontos megjegyzés a hibakezelésről

A Fetch **nem dob "rejected" Promise-t** akkor, ha a szerver 404 vagy 500 hibát ad vissza. Csak **hálózati problémák** esetén utasítja el automatikusan a Promise-t (például, ha nincs internetkapcsolat).

Ezért **mindig ellenőrizni kell a `response.ok` értékét**, és szükség esetén manuálisan kezelni a hibát.

Összefoglalás

A Fetch API a modern webfejlesztés alapvető eszköze az aszinkron adatlekéréshez. Promise-alapú működése megszünteti a **Callback Hell** problémát, és letisztult, olvasható kódot eredményez:

- Könnyebben karbantartható.
- Hibák jobban kezelhetők.
- Minden modern böngésző támogatja.